Java Arrays: Concepts, Operations, Passing and Returning Arrays

This expanded learning document introduces the fundamental concepts of arrays in Java and then develops the concepts of traversal, searching, aggregation, copying, passing arrays to methods, passing multiple arrays, and returning arrays from methods. The examples are written with explanatory comments for classroom and laboratory use.

1. What is an Array?

An array is a fixed-size collection of elements of the same data type. Instead of creating separate variables for many values, an array stores related values under one variable name. Each element is accessed using an index. Java array indexing starts from 0, so the first element is at index 0 and the last element is at index length - 1.

Example: if an array contains 5 elements, its valid indexes are 0, 1, 2, 3 and 4.

2. Why Do We Use Arrays?

• Store multiple values of the same type using a single variable.

• Process large collections of data efficiently using loops.

• Perform common operations such as searching, sorting, summation and finding minimum/maximum values.

• Pass a complete collection of values to a method using a single argument.

• Represent structured data such as marks, prices, employee IDs and sensor readings.

3. Array Declaration, Creation and Initialization

Declaration only:

```
int[] numbers;     // Declares a reference variable for an integer array
```

Creation:

```
numbers = new int[5];   // Creates an array capable of storing 5 integers
```

Declaration and creation together:

```
int[] numbers = new int[5];
```

Initialization with values:

```
int[] numbers = {10, 20, 30, 40, 50};
```

When an array is created with new, Java initializes its elements with default values. For int the default is 0, for double it is 0.0, for boolean it is false, and for reference types such as String it is null.

4. Accessing and Updating Array Elements

```
public class ArrayAccess {
    public static void main(String[] args) {

        int[] marks = {75, 82, 91, 68, 88};

        // Access the element at index 2.
        System.out.println("Third element: " + marks[2]);

        // Update the element at index 3.
        marks[3] = 72;
```

```
        System.out.println("Updated fourth element: " + marks[3]);
    }
}
```

Important: accessing an index outside the valid range causes ArrayIndexOutOfBoundsException.

5. Array length Property

The length property gives the number of elements in an array. It is commonly used in loops so that the program does not hard-code the array size.

```
int[] numbers = {10, 20, 30, 40};

System.out.println("Number of elements: " + numbers.length);

for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}
```

6. Taking Array Input from the User

```
import java.util.Scanner;

public class ArrayInput {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();

        int[] numbers = new int[n];

        // Read each element using its index.
        for (int i = 0; i < numbers.length; i++) {
            System.out.print("Enter element " + (i + 1) + ": ");
            numbers[i] = sc.nextInt();
        }

        System.out.println("Array elements:");
        for (int value : numbers) {
            System.out.print(value + " ");
        }

        sc.close();
    }
}
```

7. Traversing an Array: Traditional for vs. Enhanced for

The traditional for loop is preferred when the index is required. The enhanced for loop is convenient when only the element values are required.

```
int[] arr = {10, 20, 30, 40};

// Traditional for loop: index is available.
for (int i = 0; i < arr.length; i++) {
    System.out.println("Index " + i + " = " + arr[i]);
}

// Enhanced for loop: directly receives each value.
for (int value : arr) {
    System.out.println("Value = " + value);
}
```

8. Finding Sum, Average, Minimum and Maximum

```
public class ArrayStatistics {
    public static void main(String[] args) {

        int[] values = {10, 25, 7, 40, 18};

        int sum = 0;
        int min = values[0];
        int max = values[0];

        for (int value : values) {
            sum += value;

            if (value < min) {
                min = value;
            }

            if (value > max) {
                max = value;
            }
        }

        double average = (double) sum / values.length;

        System.out.println("Sum = " + sum);
        System.out.println("Average = " + average);
        System.out.println("Minimum = " + min);
        System.out.println("Maximum = " + max);
    }
}
```

9. Searching an Array

A simple linear search checks elements one by one until the required value is found or the array ends.

```
public class LinearSearch {
    public static void main(String[] args) {

        int[] numbers = {12, 45, 23, 67, 89, 34};
        int target = 67;

        int foundIndex = -1;

        // Check every element sequentially.
        for (int i = 0; i < numbers.length; i++) {
            if (numbers[i] == target) {
                foundIndex = i;
                break; // Stop as soon as the target is found.
            }
        }

        if (foundIndex != -1) {
            System.out.println("Element found at index " + foundIndex);
        } else {
            System.out.println("Element not found");
        }
    }
}
```

10. Copying an Array

Assigning one array variable to another does not create a new array. Both variables refer to the same array object. To create an independent copy, use a copying technique such as Arrays.copyOf().

```
import java.util.Arrays;

public class ArrayCopy {
    public static void main(String[] args) {

        int[] original = {10, 20, 30, 40};

        // Creates a new array containing the same values.
        int[] copy = Arrays.copyOf(original, original.length);

        // Modify the copy.
        copy[0] = 999;

        System.out.println("Original first element: " + original[0]);
        System.out.println("Copy first element: " + copy[0]);
    }
}
```

11. Sorting an Array

```
import java.util.Arrays;

public class ArraySorting {
    public static void main(String[] args) {

        int[] numbers = {50, 10, 40, 20, 30};

        // Sorts the array in ascending order.
        Arrays.sort(numbers);

        System.out.println(Arrays.toString(numbers));
    }
}
```

12. Passing an Array to a Method

Java allows an array to be passed as an argument to a method. The parameter is declared using array syntax such as int[]. The method can read the array and, if required, modify its elements.

```
public class ArrayMethodPass {

    // Receives an array and returns the sum of its elements.
    static int findSum(int[] array) {

        int sum = 0;

        for (int i = 0; i < array.length; i++) {
            sum += array[i];
        }

        return sum;
    }

    public static void main(String[] args) {

        int[] numbers = {1, 2, 3, 4, 5};

        // Pass the complete array to the method.
        int result = findSum(numbers);

        System.out.println("The sum is: " + result);
    }
}
```

13. Passing Multiple Arrays to a Method

```
public class MultipleArrayPassing {

    // Adds corresponding elements of two arrays.
    static void sumArrays(int[] arr1, int[] arr2) {

        // Both arrays should have compatible lengths.
        for (int i = 0; i < arr1.length; i++) {
            System.out.println(
                arr1[i] + " + " + arr2[i]
                + " = " + (arr1[i] + arr2[i])
            );
        }
    }

    public static void main(String[] args) {

        int[] arr1 = {1, 2, 3, 4, 5};
        int[] arr2 = {6, 7, 8, 9, 1};

        sumArrays(arr1, arr2);
    }
}
```

14. Array Reference and Modification

An array variable holds a reference to an array object. When that array is supplied to a method, the method receives a copy of the reference to the same array object. Consequently, changing an element through the method is visible through the original array variable.

```
public class ArrayReferenceDemo {

    static void changeFirstElement(int[] array) {
        // Modify the original array object.
        array[0] = 100;
    }
```

```
    public static void main(String[] args) {

        int[] numbers = {10, 20, 30};

        System.out.println("Before: " + numbers[0]);

        changeFirstElement(numbers);

        System.out.println("After: " + numbers[0]);
    }
}
```

15. Passing Arrays to a Non-Static Method

A non-static method belongs to an object. Therefore, an object of the class is created before the method is invoked from the static main() method.

```
public class ArrayWithoutStatic {

    // This method is non-static.
    void addArrays(int[] a, int[] b) {

        for (int i = 0; i < a.length; i++) {
            System.out.println(
                a[i] + " + " + b[i]
                + " = " + (a[i] + b[i])
            );
        }
    }

    public static void main(String[] args) {

        int[] a = {10, 20, 30};
        int[] b = {1, 2, 3};

        // Create an object to call the non-static method.
        ArrayWithoutStatic obj = new ArrayWithoutStatic();

        obj.addArrays(a, b);
    }
}
```

16. Returning an Array from a Method

A method can return an entire array. The return type includes brackets, for example int[]. The caller can store the returned array in another reference variable.

```
public class ReturnArray {

    // Creates and returns an integer array.
    static int[] createArray() {

        int[] arr = {10, 20, 30, 40};

        return arr;
    }

    public static void main(String[] args) {

        // Receive the array returned by the method.
        int[] numbers = createArray();

        for (int value : numbers) {
            System.out.print(value + " ");
        }
    }
}
```

17. Modifying and Returning an Array

```
public class ReturnDoubleArray {

    // Doubles each element and returns the same array reference.
    static int[] doubleArray(int[] array) {

        for (int i = 0; i < array.length; i++) {
            array[i] = array[i] * 2;
        }

        return array;
```

```
    }

    public static void main(String[] args) {

        int[] arr = {1, 2, 3, 4, 5};

        // Pass the array to the method.
        int[] result = doubleArray(arr);

        System.out.println("Doubled array:");

        for (int value : result) {
            System.out.print(value + " ");
        }
    }
}
```

18. Two-Dimensional Arrays

A two-dimensional array can be used to represent data in rows and columns, such as a matrix or a marks table. It is essentially an array whose elements are themselves arrays.

```
public class TwoDArray {
    public static void main(String[] args) {

        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        // Outer loop processes rows.
        for (int i = 0; i < matrix.length; i++) {

            // Inner loop processes columns.
            for (int j = 0; j < matrix[i].length; j++) {
                System.out.print(matrix[i][j] + " ");
            }

            System.out.println();
        }
    }
}
```

19. Practical Example: Find the Second Largest Element

```
public class SecondLargest {
    public static void main(String[] args) {

        int[] numbers = {12, 45, 23, 67, 34, 67, 50};

        int largest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;

        for (int value : numbers) {

            if (value > largest) {
                // Previous largest becomes second largest.
                secondLargest = largest;
                largest = value;
            } else if (value > secondLargest && value != largest) {
                secondLargest = value;
            }
        }

        System.out.println("Largest = " + largest);
        System.out.println("Second Largest = " + secondLargest);
    }
}
```

20. Important Concepts to Remember

• Array size is fixed after creation.

• Array indexing starts at 0.

• The last valid index is array.length - 1.

• The length property gives the number of elements.

• All elements of a normal Java array have the same declared type.

• The traditional for loop is useful when the index is needed.

• The enhanced for loop is convenient when only values are needed.

• An array can be passed to a method using a parameter such as int[].

• A method can return an array using a return type such as int[].

• Changing an element inside a method can affect the same array object visible to the caller.

• Arrays.copyOf() can be used when an independent copy is required.

• Arrays.sort() can sort a one-dimensional array in ascending order.

• Two-dimensional arrays are useful for row-column data.

21. Suggested Laboratory Exercises

• Read n integers into an array and display them in reverse order.

• Find the sum, average, minimum and maximum of an integer array.

• Search for an element using linear search and display its index.

• Count even and odd numbers in an array.

• Count positive, negative and zero values.

• Copy an array and demonstrate the difference between reference assignment and an independent copy.

• Pass an array to a method that returns its sum.

• Pass two arrays to a method and generate their element-wise sum.

• Write a method that returns a reversed array.

• Write a method that returns the second-largest element.

• Create a two-dimensional array and calculate the sum of each row and each column.

• Sort an array and then perform binary search on the sorted array.